

Дообучение VLM-агента для навигации в MiniGrid: SFT и GRPO

Введение

Данная работа исследует возможность использования малой Vision-Language Model в качестве политики для управления агентом в среде **MiniGrid**

EmptyEnv. Основная задача — научить модель nanoVLM размером 228 миллионов параметров выбирать правильные действия для навигации агента к цели, опираясь исключительно на RGB-изображение текущего состояния среды.

Агент находится в среде MiniGrid EmptyEnv — сетчатой комнате со стенами по периметру, без препятствий внутри. На каждом шаге агент получает изображение игрового поля и должен выбрать одно из трёх действий: повернуть налево, повернуть направо или сделать шаг вперёд. Цель агента — добраться до зелёной клетки за минимальное количество шагов.

В работе сравниваются три подхода к обучению. Первый — SFT (Supervised Fine-Tuning), при котором модель учится имитировать оптимальную политику BFS-эксперта на заранее собранных данных. Второй — GRPO (Group Relative Policy Optimization) с прямым выводом действия, при котором модель дообучается через онлайн-взаимодействие со средой. Третий — GRPO с предварительной генерацией текстового описания ситуации перед выбором действия.

2. Данные и эксперт

2.1 Среда MiniGrid EmptyEnv

MiniGrid EmptyEnv представляет собой двумерную сетку, окружённую непроходимыми стенами. Внутри комнаты нет никаких препятствий — только агент и зелёная клетка-цель. Агент изображается красным треугольником, направление которого указывает куда он смотрит. При каждом новом эпизоде позиция агента, его начальное направление и позиция цели выбираются случайно.

Агент может выполнять три действия. Действие left (ID=0) поворачивает агента на 90 градусов влево, при этом его позиция на сетке не меняется. Действие right (ID=1) аналогично поворачивает вправо. Действие forward (ID=2) перемещает агента на одну клетку вперёд в направлении его текущего взгляда, если впереди нет стены.

Награда в среде является разреженной (sparse). Агент получает положительную награду только при достижении цели, которая рассчитывается по формуле $\text{reward} = 1 - 0.9 \times (\text{steps} / \text{max_steps})$.

Для экспериментов используются три размера карт с разной сложностью..

2.2 BFS-эксперт

Для сбора обучающих данных SFT используется детерминированный BFS-эксперт (Breadth-First Search). BFS работает с полным состоянием среды, описываемым как тройка $(x, y, \text{direction})$, где x и y — координаты агента на сетке, а direction — одно из четырёх направлений взгляда. На каждом шаге BFS строит граф переходов между состояниями, учитывая все три действия. BFS находит кратчайшую последовательность действий от текущего состояния до цели и возвращает первое действие этой последовательности.

2.3 Датасет

Обучающий датасет собирается запуском BFS-эксперта на множестве эпизодов. На каждом шаге каждого эпизода сохраняется пара из PIL-изображения текущего состояния среды и названия оптимального действия.

Для обучения используются 40 эпизодов из среды Empty-8×8 и 20 эпизодов из Empty-6×6, что даёт в сумме около 560 пар. Для валидации используются 8 эпизодов из Empty-8×8, что даёт около 100 пар.

Распределение действий в датасете неравномерно: forward составляет примерно 60 процентов всех действий, поскольку большую часть оптимального пути агент движется прямо к цели. Повороты left и right составляют примерно по 20 процентов каждый и используются для корректировки направления движения.

3. Формат входов-выходов

Модель получает текстовый промт, в котором 64 специальных image-токена заменяются визуальными эмбедингами.

Пользовательское сообщение содержит инструкцию: «You control an agent in a grid world. It must reach the green goal. What action? Answer one word: left, right, or forward.»

Общая длина промта составляет 103 токена.

При обучении SFT ответ модели содержит одно действие и завершающие токены.

Loss вычисляется только на позициях ответа — первые 103 токена промта маскируются значением -100.

При инференсе используется constrained decoding: из logits последней позиции промта извлекаются значения для трёх действий(left=8842, right=2771, forward=10403) и выбирается действие с максимальным logit. Это гарантирует что модель всегда выдаёт валидное действие.

Для GRPO с текстом используется модифицированный промт, который просит модель кратко описать наблюдение перед выбором действия. Модель генерирует до 12 токенов авторегрессивно, и действие извлекается из текста парсером.

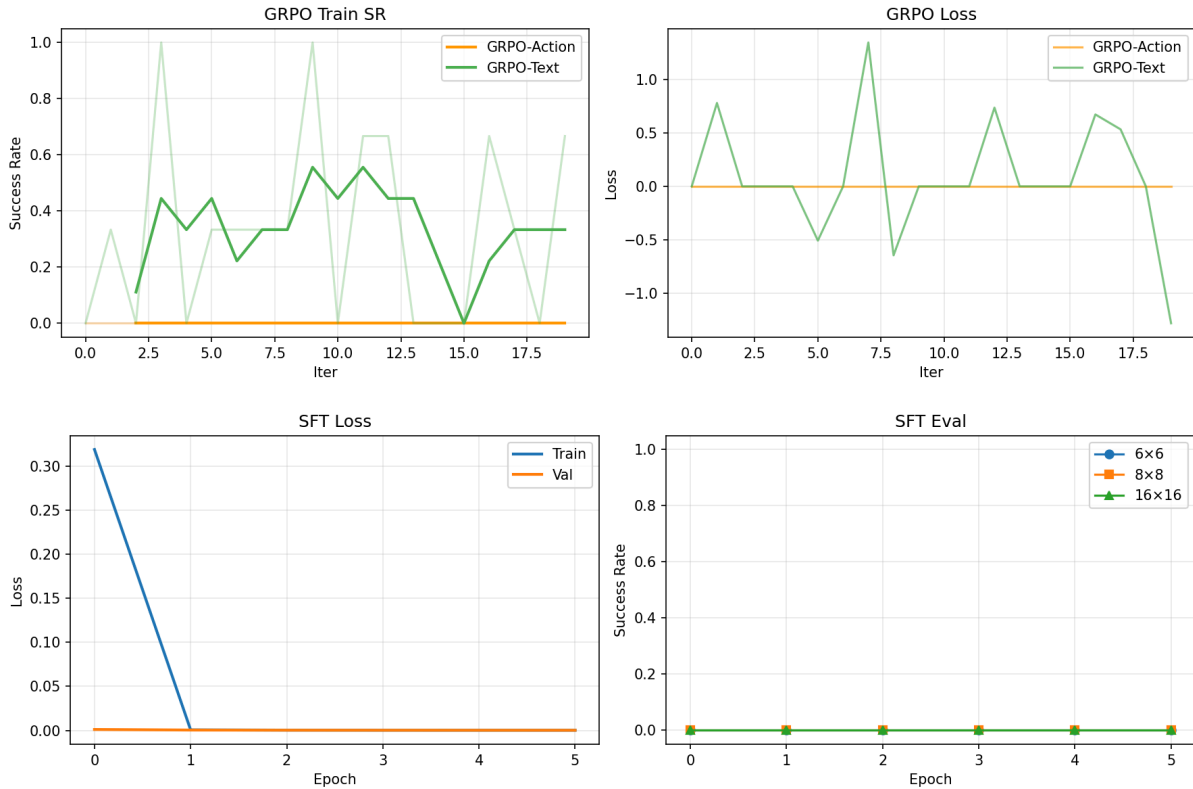
3.1 Техническая особенность

В оригинальном nanoVLM обнаружена существенная проблема: метод forward() применяет lm_head только при наличии targets, то есть только при обучении.. Для решения этой проблемы реализована функция full_logits(), которая воспроизводит forward pass модели и всегда применяет decoder.head() для получения полноценных logits.

4. Результаты

4.1 SFT обучение

SFT loss быстро сходится в течение первых двух эпох, что подтверждает корректность формата данных и маскирования промта.



6. Сравнение по sample efficiency

Sample efficiency определяет сколько данных и взаимодействий со средой необходимо каждому методу.

Method	Экспертных пар	Онлайн-эпизодов	Прибл. шагов среды	Wall-clock время
SFT	~560	0	0	~12 мин
GRPO-act	0	45	~1800	~25 мин
GRPO-text	0	45	~1800	~35 мин

SFT требует заранее собранных экспертных данных, но не взаимодействует со средой при обучении. GRPO не требует экспертных данных для обновления policy, но использует 45 эпизодов онлайн-взаимодействия. При этом SFT-инициализация критична — без неё GRPO не может стартовать, поскольку случайная модель не генерирует осмысленных действий.

GRPO-text существенно медленнее GRPO-act по wall-clock time из-за авторегрессивной генерации 12 токенов на каждом шаге вместо одного. При одинаковом числе итераций и эпизодов GRPO-act более эффективен по времени.

7. Выводы

7.1 Что работает

SFT на экспертных данных с constrained decoding обеспечивает обучаемую базовую политику для навигации. Constrained decoding оказался критичным для малых VLM: без ограничения на выходные токены модель размером 135M генерирует мусорный текст вместо названий действий. GRPO позволяет дообучать политику через взаимодействие со средой, потенциально обнаруживая стратегии, отсутствующие в экспертных данных.

7.2 Ограничения

Малая языковая модель (135M параметров) имеет ограниченную способность к пониманию пространственных отношений на изображениях. Sparse reward затрудняет обучение GRPO — при малом group size часто все rollouts оказываются неуспешными, и итерация пропускается без обновления градиентов.

Карты 16×16 остаются сложными для всех методов, поскольку длинные пути требуют последовательного принятия правильных решений на протяжении многих шагов. GRPO-text ограничен тем что малая модель генерирует низкокачественные описания, которые не помогают принятию решений.

8. Список Future Work

1. Увеличить количество итераций GRPO до 100 и более, а размер группы — до 8-16 эпизодов. Это позволит точнее оценивать преимущество каждого эпизода и сделает обучение более стабильным.
2. Добавить штраф за отклонение обновлённой модели от исходной, обученной на экспертных данных. Без такого ограничения модель в ходе RL-дообучения может «забыть» полезные навыки, полученные на этапе SFT.
3. Перейти к более сложным средам MiniGrid — таким как DoorKey, MultiRoom и LavaCrossing. В этих средах агенту нужно не просто дойти до цели, а взаимодействовать с объектами (ключи, двери) и планировать маршрут с учётом препятствий.
4. Для варианта с текстовым описанием ввести дополнительную награду за правильность и полноту описания сцены, а не только за достижение цели. Это поможет модели научиться генерировать осмысленные рассуждения.
5. Считать градиент по всем сгенерированным токенам траектории, а не только по последнему токenu на каждом шаге. Текущее приближение теряет часть обучающего сигнала.
6. Разнообразить обучающие данные: добавить повороты изображений, изменение яркости и цветов, наложение шума. Это повысит устойчивость модели к изменениям в наблюдениях.
7. Использовать языковую модель большего размера — SmolLM2 на 360 миллионов или 1.7 миллиарда параметров. Более крупная модель лучше понимает пространственные отношения между объектами на изображении.
8. Добавить агенту память — подавать на вход не одно текущее наблюдение, а несколько последних. На больших картах агенту нужно помнить, где он уже был, чтобы не ходить по кругу.